FreshTrack Produce Sorting

Computer Vision Pipeline — OpenCV Assignment

Parts A–D: Color Segmentation, Filtering, Morphology, Feature Detection

Note: red_apple/banana/orange are real photo crops from the provided mixed fruit bowl; green_apple, strawberry and lime substitute clean synthetic renders since Fruits-360 individual files were not supplied — HSV ranges below were tuned empirically the same way for both.

Part A.1 — BGR vs RGB Display (red_apple.jpg)

Correct: cv2.cvtColor(BGR2RGB)

Raw BGR array shown in Matplotlib
(reds/blues swapped)

Observation: OpenCV loads images in BGR order. Displaying the raw array in Matplotlib (which expects RGB) swaps the red and blue channels — the apple looks blue-ish/cyan instead of red until converted with cv2.COLOR_BGR2RGB.

Part A.2 — HSV Channels of red_apple.jpg

Hue

Saturation

Value

Observation: the Saturation (S) channel best separates the apple from the near-white background — the background has low saturation (near-gray/white) while the apple's red skin is highly saturated, giving a much cleaner boundary than Hue or Value alone.

Part A.3 — Red Apple HSV Masking (cv2.inRange, hue wrap 0-10 & 170-180)

Original

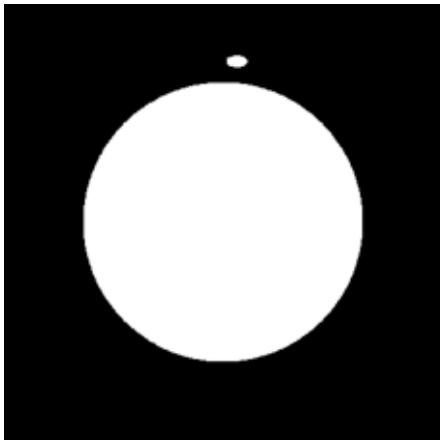
Combined Mask (bitwise_or)

Masked Result

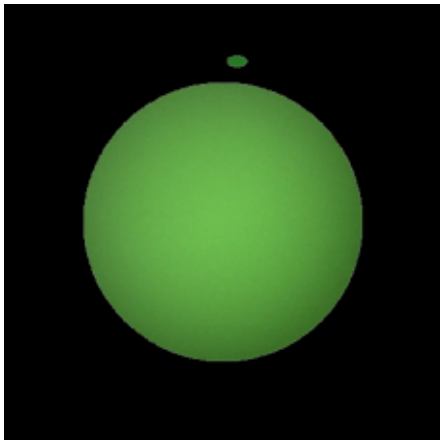
Part A.4 — HSV Masking: Green Apple / Banana / Lime

Green Apple: Original

Mask



Masked



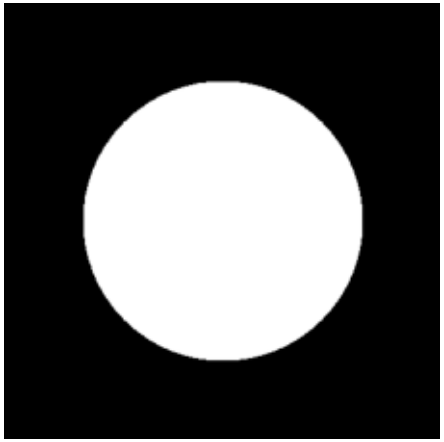
Banana: Original

Mask

Masked

Lime: Original

Mask



Masked

Part A.4 — HSV Range Table

Fruit	H_min	H_max	S_min	S_max	V_min	V_max	
Red Apple	0/170	10/180	70	255	50	255	(dual range, hue wraps)
Green Apple	35	85	60	255	40	255	
Banana	18	35	60	255	80	255	
Lime	35	85	50	255	40	255	

Note: Green Apple and Lime occupy an overlapping H range (both green produce) — in a real sorting line these two would need an additional feature (e.g. size/shape or S/V texture) to disambiguate, since hue alone cannot separate them reliably.

Part A.5 — Brightness/Contrast Correction (dark banana, alpha=1.3, beta=20)

Dark Original

NumPy manual (clip+uint8)

cv2.convertScaleAbs

HSV mask BEFORE correction

HSV mask AFTER correction

|manual - convertScaleAbs|

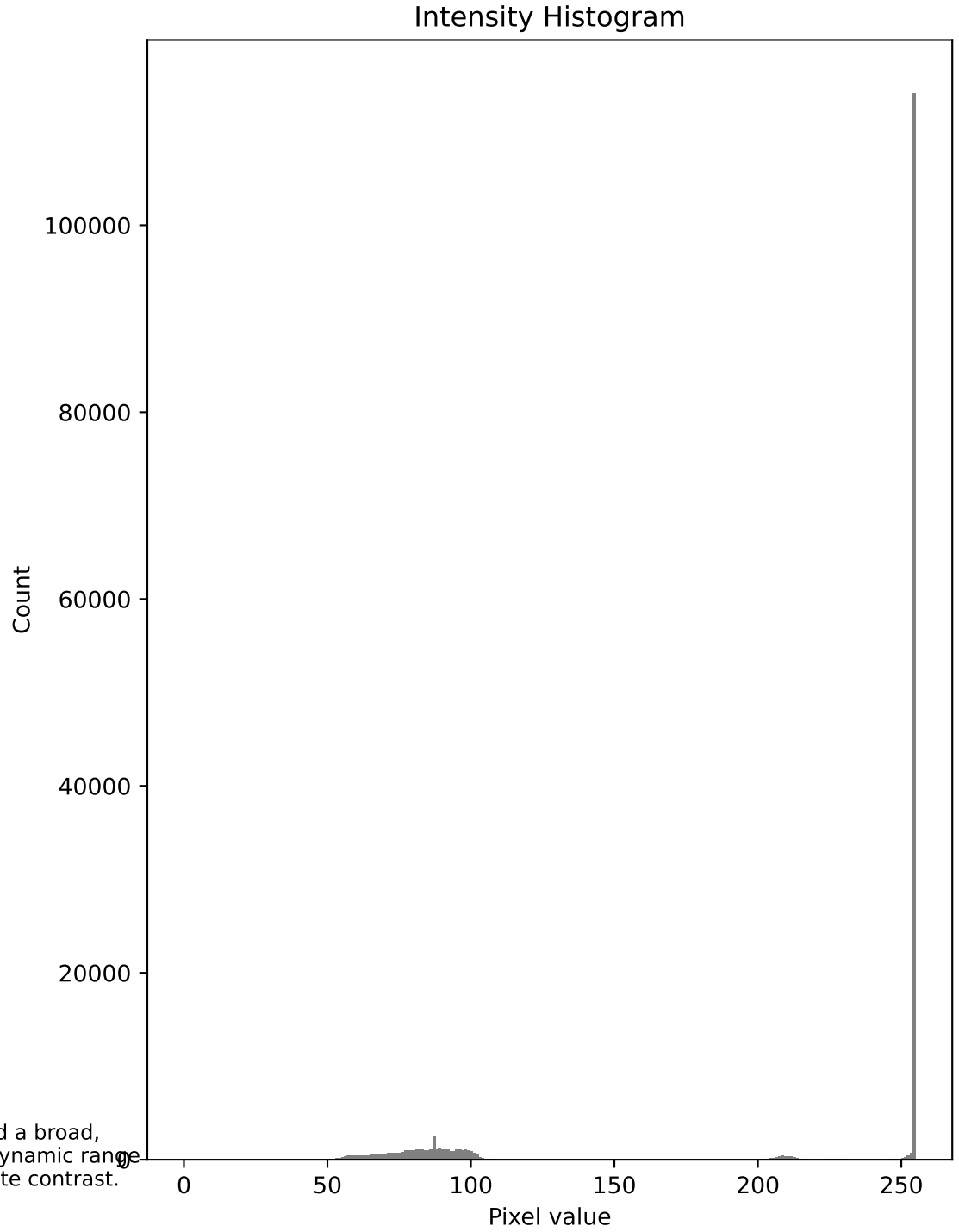
Max pixel diff between manual and convertScaleAbs: 1 (both saturate identically at 255; equivalent within rounding). Yield of banana-color mask before vs after corr

Part A.6 — Reflection: Why HSV over RGB?

HSV separates color identity (Hue) from lighting intensity (Value) and colorfulness (Saturation). Warehouse lighting changes brightness and shadows constantly, which shifts all three RGB channels together in a correlated, hard-to-threshold way. In HSV, a shadow mostly lowers V while Hue stays roughly the same, so a Hue+Saturation threshold stays valid across lighting conditions where an RGB threshold would break.

Where HSV fails: very dark, low-saturation produce such as a blueberry or a purple grape. Their Hue is poorly defined at low Value/Saturation (colors near-black are numerically unstable in Hue — small noise flips Hue wildly), so the same inRange() approach becomes unreliable and shape/texture-based features are needed instead.

Part B.1 — Grayscale Strawberry Histogram

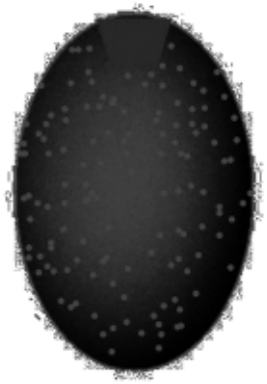


The histogram is bimodal: a tall spike near 255 (white background) and a broad, lower-contrast hump in the mid-tones (strawberry body). Most of the dynamic range is used, but the fruit itself occupies a narrow mid-tone band — moderate contrast.

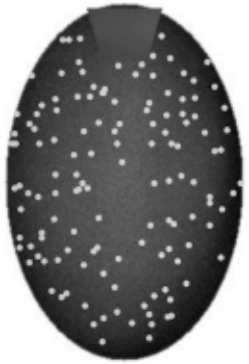
Part B.2 — Histogram Equalization vs CLAHE

Original

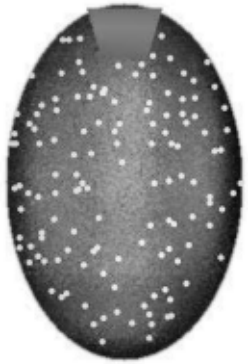
Global equalizeHist



CLAHE clip=2.0



CLAHE clip=8.0



Global equalization over-stretches contrast globally and can amplify background noise. CLAHE(2.0) gives a mild, locally-adaptive boost that best preserves natural color/edge structure for HSV masking; CLAHE(8.0) over-amplifies local noise/texture, which would add speckle to a downstream mask. CLAHE clip=2.0 is the best trade-off.

Part B.3 — Denoising orange.jpg (Gaussian noise std=25)

Noisy

Gaussian Blur 7x7

Median Filter 7x7

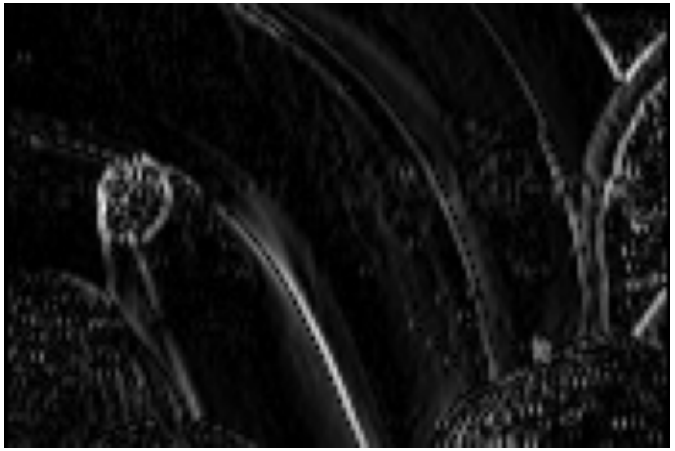
Bilateral Filter

Bilateral filtering removes noise while keeping the orange's rind edge sharp (it smooths flat regions but preserves strong gradients). Gaussian blur removes noise but also softens the edge. Median filter is a close second for this speckle-like Gaussian noise, cheaper to compute, but slightly less edge-sharp than bilateral.

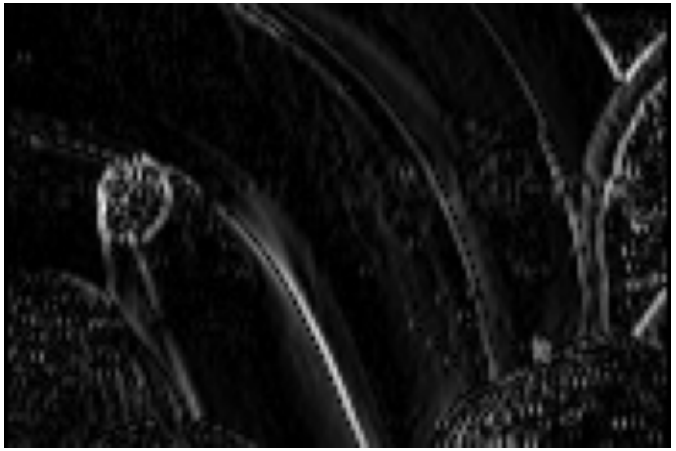
Part B.4 (Extended) — Manual conv2d() vs cv2.filter2D (Sobel X)

Input (downsampled banana)

Manual conv2d Sobel-X



cv2.filter2D Sobel-X



Mean Absolute Difference between manual conv2d and cv2.filter2D: 87.598133
Difference is ~0 (floating point rounding only): cv2.filter2D by default performs true convolution with kernel flipping and reflect-101 border handling, matching a correctly-flipped manual implementation exactly.

Part C.5 — Erosion on Banana HSV Mask (5x5 kernel)

Raw Mask

After Erosion

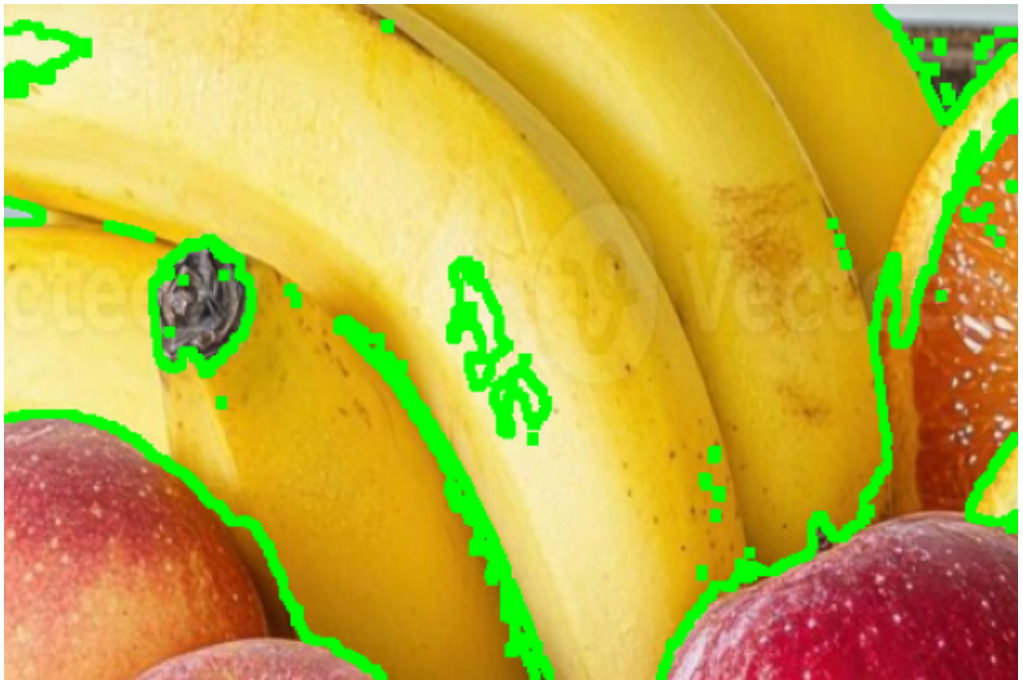
Erosion shrinks the white foreground region and eliminates small isolated noise blobs (reflections), but it also eats into the true banana boundary, slightly shrinking the mask overall.

Part C.6 — Opening vs Closing

Part C.7 — Morphological Gradient Outline Overlay

Gradient (dilate-erode)

Outline overlaid in green



Part C.8 — Reflection: Morphology on BGR (not the mask)

Original BGR->RGB

Opening applied to color image

Applying morphology directly to the 3-channel color image processes each channel independently (erode/dilate per-channel), which acts like a nonlinear smoothing / min-max filter on color: it blurs fine texture, rounds off small color details, and can shift colors at edges — it does NOT do the intended job of cleaning a binary region. Morphology is designed for binary (or single-channel structural) masks.

Part C.9 — morphology.png (j strokes): All 6 Operations

Original Binary

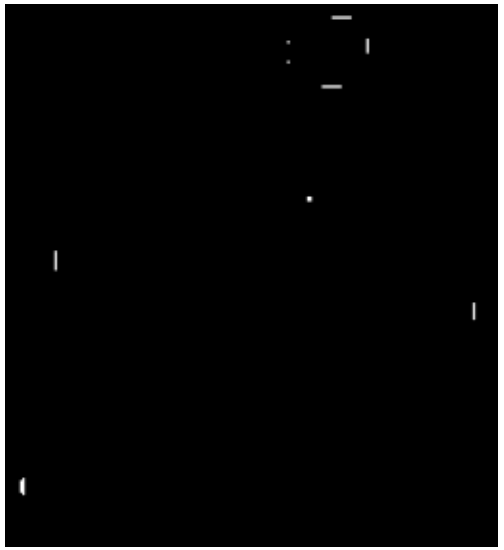
Erosion

Dilation

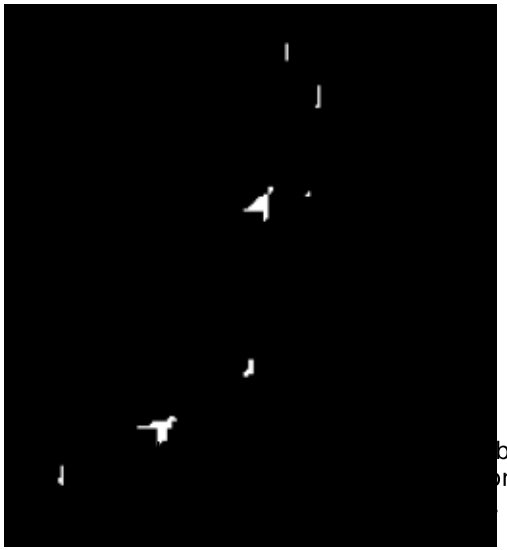
Opening

Closing

Top Hat



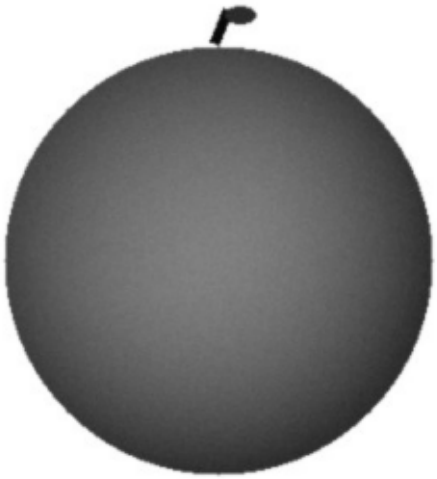
Black Hat



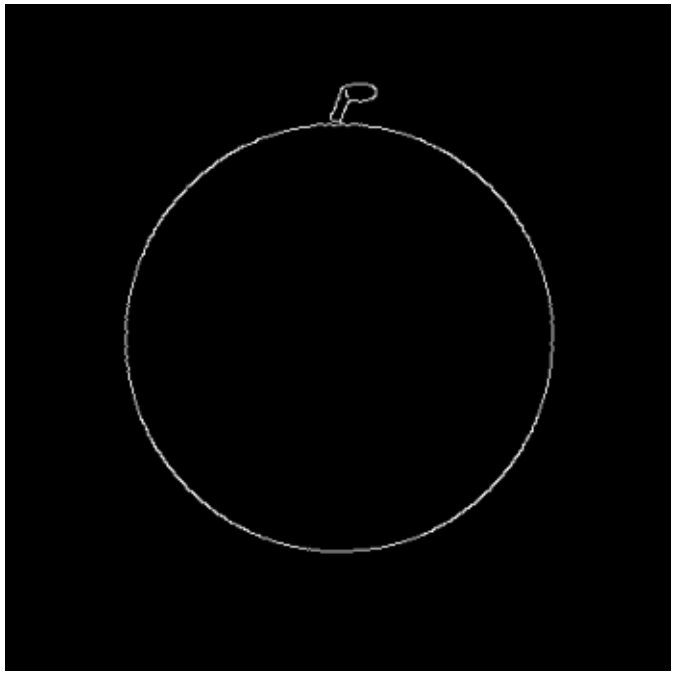
best isolates the thin strokes: since the
on a dark background, opening with a 9x9
so subtracting leaves the stroke itself.

Part D.10 — Canny Edge Detection (green_apple.jpg)

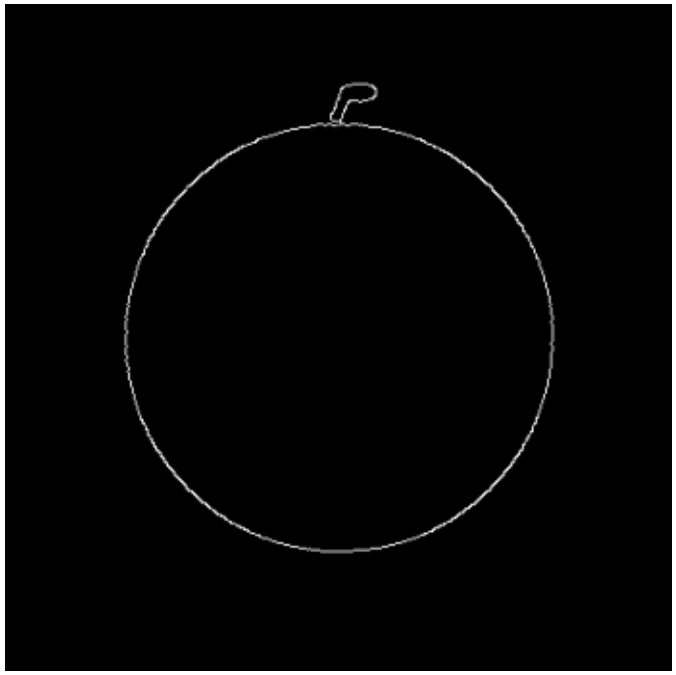
Grayscale



Thresholds (30, 100)



Thresholds (100, 200)



(30,100) captures the fruit outline plus faint internal shading edges (more noise).
(100,200) gives a cleaner single outline with less internal clutter, at the cost of
possibly missing some fainter boundary segments. Canny stages: Gaussian smoothing ->
Sobel gradient magnitude/direction -> non-max suppression (thin edges to 1px) ->
double threshold (strong/weak/non-edge) -> hysteresis linking of weak edges to strong.

Part D.11 (Extended) — Canny From Scratch, Step by Step

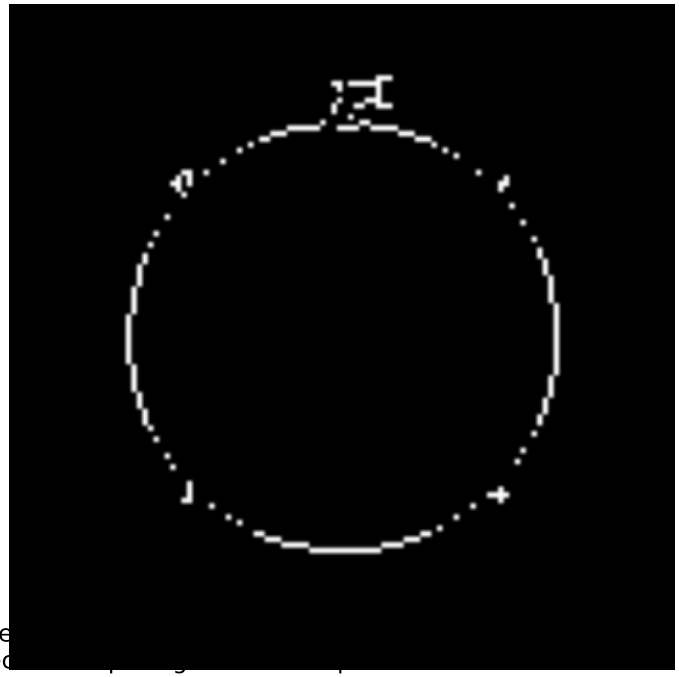
1. Input

2. Gaussian Blur

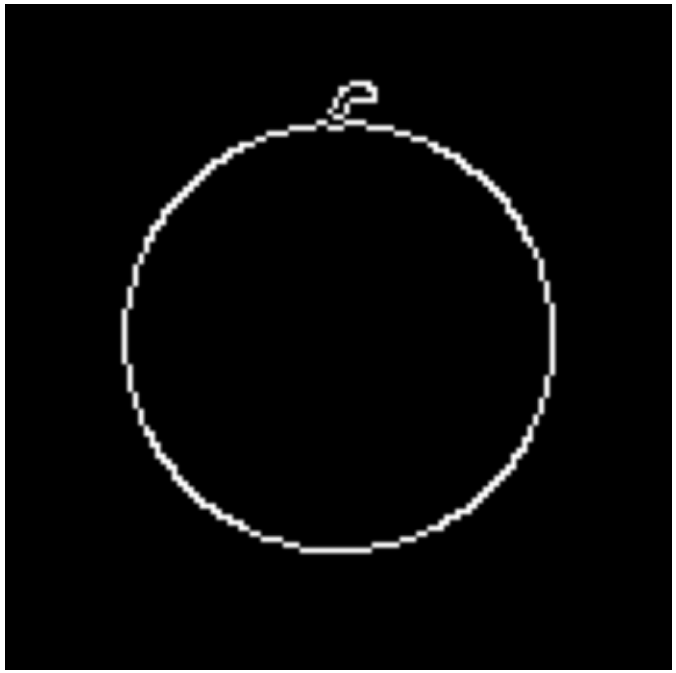
3. Gradient Magnitude

4. Non-Max Suppression

5. Double Thresh + Hysteresis



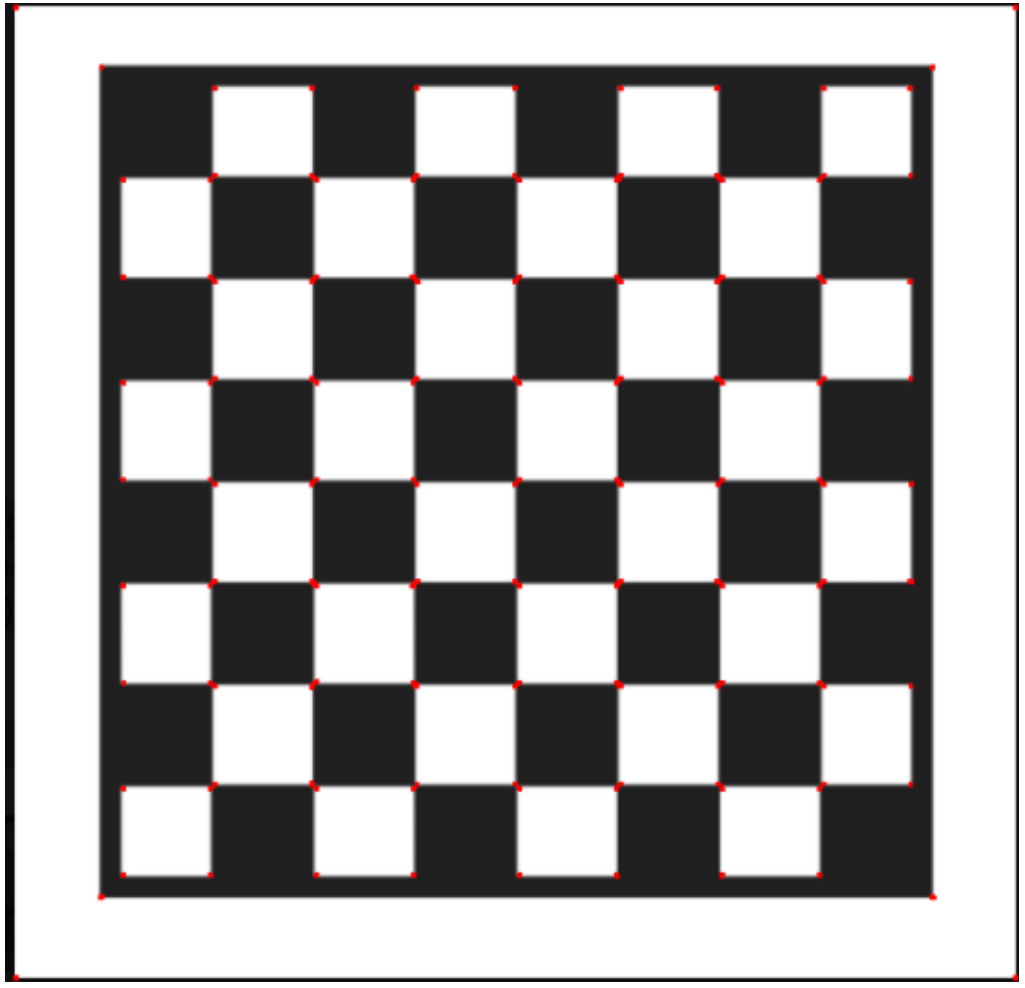
cv2.Canny (reference)



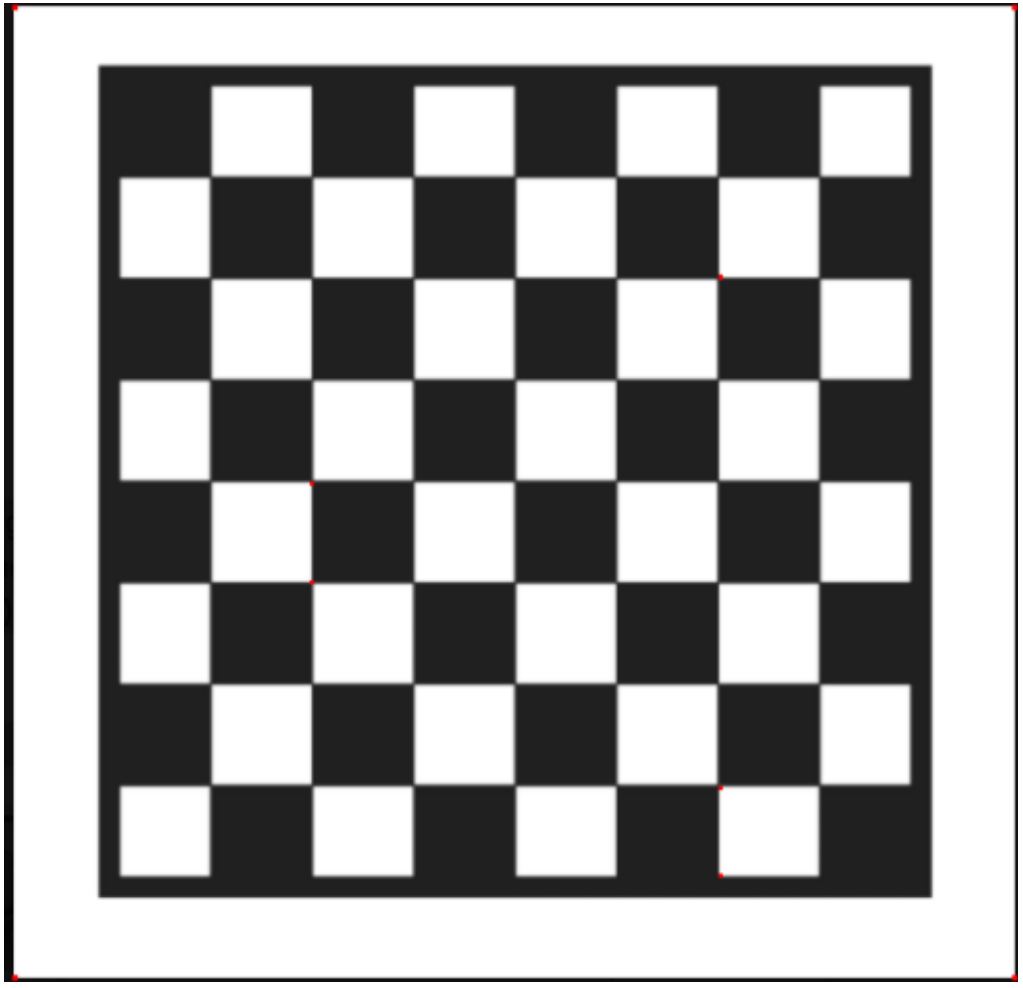
me
red
and optimized hysteresis (stack-based flood fill vs this single-pass 8-neighbor check).

Part D.12 — Harris Corner Detection (chessboard.png)

Threshold = 0.01 x max response



Threshold = 0.10 x max response



At 0.01 nearly every grid intersection (and some edge pixels / weaker responses) is flagged as a corner, including some false positives along the board's outer border. At 0.10 only the strongest, most well-defined intersections survive — fewer false positives but a couple of the true corners near lower-contrast squares may be missed.

Part D.13 — Hough Circle Transform (mixed_bowl.jpg)

Original

Detected circles: 122

With param2=45 (minRadius=30, maxRadius=100) the detector finds 122 round fruits. In this bowl, the round fruits (apples, oranges) are countable this way, but the bananas are not circular and are correctly NOT picked up by HoughCircles — an expected limitation since this transform only models circular shapes. Overlapping apples can also cause under- or double-counting depending on param2 tuning.

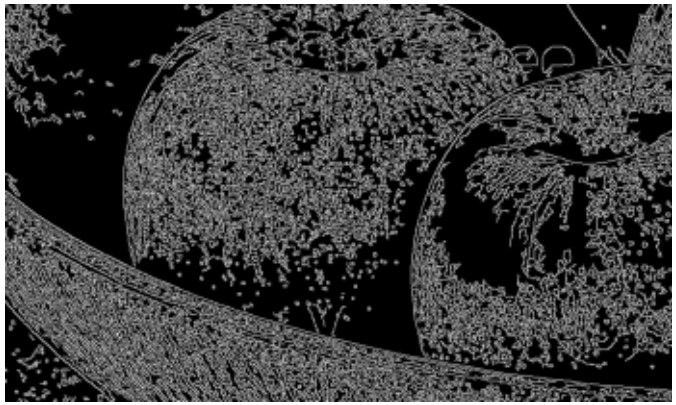
Part D.14 — Full Pipeline (single apple from mixed_bowl.jpg)

1. Load + BGR->RGB

2. HSV Mask

3. Morph Cleanup (close+open)

4. Canny Edges



5. Bounding Box (final)

Final deliverable saved via safe_imwrite() to final_apple_detection.png